Lumina AI – Technical Documentation

Multi-Agent Retrieval-Augmented Generation (RAG) Knowledge Retrieval Platform

> Items marked **[Verify]** describe a design intent or recommended default that must be confirmed against the actual codebase before submission. Nothing here should be reported as implemented until observed in the running application.

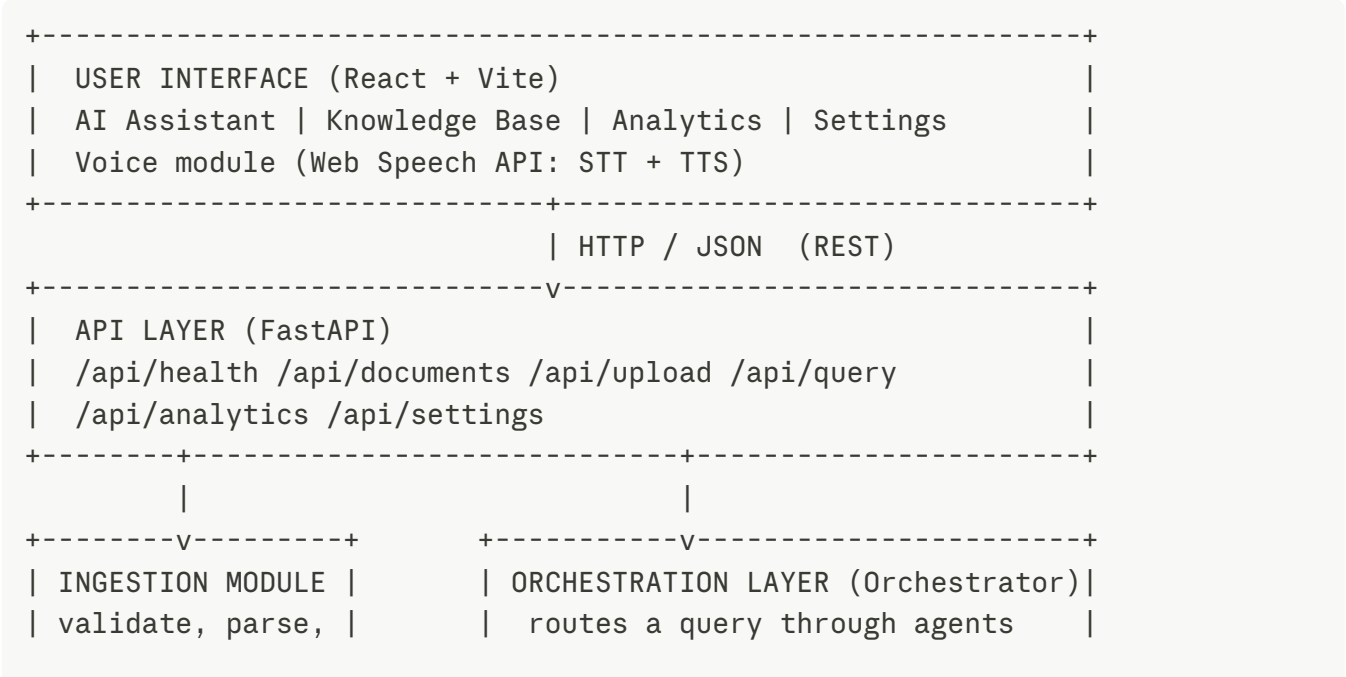
1. Introduction

Lumina AI lets users upload documents into a knowledge base and ask natural-language questions about them. A document-ingestion pipeline extracts, cleans, chunks, embeds and indexes content. A multi-agent query pipeline classifies each question, retrieves relevant chunks, generates a source-grounded answer, asks for clarification when a question is ambiguous, and remembers conversational context. The interface also offers voice input, text-to-speech, response transparency and an analytics dashboard that exposes unanswered questions and knowledge gaps.

Scope. Supported formats are PDF, DOCX, TXT and CSV. The local implementation uses a React/Vite frontend and a Python FastAPI backend, with the Google Gemini API for language-model calls and the Web Speech API for voice.

2. System Architecture

2.1 Layered view



```
| clean, chunk,    |       |                                    |
| embed, index     |       |  Query Understanding Agent         |
+--------+---------+       |  Clarification Agent               |
         |                 |  Conversation Memory Agent         |
         |                 |  Retrieval Agent                   |
         |                 |  Response Generation Agent         |
         |                 +-----------+----------------------+
         |                             |
+--------v-----------------------------v----------------------+
|  STORAGE                                                    |
|  Vector store (chunk embeddings + metadata)                 |
|  Metadata DB (documents, queries, sessions, analytics)      |
|  File storage (original uploads)                            |
+-----------------------------+-------------------------------+
                              |
+-----------------------------v-------------------------------+
|  EXTERNAL: Google Gemini API (generation; embeddings [Verify])|
+-------------------------------------------------------------+
```

2.2 Component summary

Component	Role	Key responsibilities
Knowledge Base Ingestion Module	Turns files into searchable chunks	Validation, text extraction, cleaning, chunking, embedding, indexing, metadata storage
Vector store	Semantic search index	Stores chunk vectors and metadata, returns Top-K nearest chunks with scores
Metadata database	Records	Documents, chunks, query logs, sessions, analytics aggregates
Orchestration layer	Pipeline controller	Calls agents in order, handles branching (clarify vs. retrieve), error handling, timing
Agents (5)	Specialized tasks	See Section 3
Conversation memory	Session context	Recent turns, active topic, referenced entities; separate from the document knowledge base
Voice modules	Speech I/O	Speech-to-text input, text-to-speech output via the Web Speech API

Component	Role	Key responsibilities
Analytics module	Usage insight	Query logging, category counts, latency, low-confidence and unanswered queries, knowledge gaps
User interface	Interaction	Chat, uploads, sources panel, analytics dashboard, settings, loading and error states

2.3 Design principles

1. **Grounding first.** Document-specific answers must come from retrieved chunks. If evidence is missing, the system says so.
2. **Separation of concerns.** Each agent has one job and a structured input/output contract.
3. **Transparency.** Every answer carries the sources that were actually retrieved for it.
4. **Honest analytics.** Metrics come from stored activity, never from sample counters.
5. **Memory is not knowledge.** Conversation memory is temporary session context and is never written to the document index.

3. Agents: Responsibilities and Interaction Flow

3.1 Query Understanding Agent

- **Purpose:** interpret the question and decide how to process it.
- **Input:** raw user query, recent conversation context (from the Memory Agent).
- **Output (structured):**

```
{
  "query_type": "factual | procedural | comparative | ambiguous",
  "intent": "short description",
  "entities": ["NDVI", "SMAP"],
  "is_follow_up": true,
  "needs_clarification": false,
  "rewritten_query": "standalone version of the question"
}
```

- **Behaviour:** "What is NLP?" is factual. "How do I upload a document?" is procedural. "Compare symmetric and asymmetric encryption" is comparative. "What about memory?" with no usable context is ambiguous.

- **Failure handling:** if the model returns malformed JSON, fall back to a default classification (factual, no clarification) and log a warning.

3.2 Retrieval Agent

- **Purpose:** find the most relevant evidence in the knowledge base.
- **Input:** the rewritten query and retrieval settings (Top-K, threshold, optional document filter).
- **Process:** embed the query, run vector similarity search, take Top-K, drop chunks below the relevance threshold, keep metadata.
- **Output:**

```
{
  "chunks": [
    {"chunk_id": "c_0412", "document_id": "d_07", "document_name":
"smap_vod.pdf",
     "page": 5, "text": "...", "score": 0.78}
  ],
  "max_score": 0.78,
  "found_evidence": true
}
```

- **No-evidence case:** if no chunk survives the threshold, `found_evidence = false`; the orchestrator routes to the "cannot find in documents" response and logs the query as unanswered.

3.3 Response Generation Agent

- **Purpose:** write the answer from the retrieved evidence.
- **Input:** question, query type, retrieved chunks, brief conversation context.
- **Prompt contract:** answer only from the supplied context for document-specific questions; do not invent facts; state clearly when the context is insufficient; adjust format to query type (steps for procedural, side-by-side for comparative).
- **Output:** answer text, list of chunk IDs actually used, and a confidence/retrieval-quality indicator.
- **Confidence note:** the indicator is derived from retrieval scores and chunk coverage. It indicates retrieval quality, not guaranteed correctness.

3.4 Clarification Agent

- **Purpose:** resolve ambiguous or incomplete questions instead of guessing.

- **Trigger:** `needs_clarification = true`, or `query_type = ambiguous`, or very low retrieval confidence with several plausible topics.
- **Behaviour:** asks one focused question, optionally offering choices drawn from the knowledge base (for example, "Do you mean computer memory, conversation memory, or memory in the RAG pipeline?").
- **Resume:** when the user answers, the orchestrator merges the clarification with the original query and re-enters the pipeline at classification.
- **Safeguard:** limit to one clarification round per query; after that, answer with best-effort evidence and state what was assumed.

3.5 Conversation Memory Agent

- **Purpose:** maintain relevant context across turns.
- **State per session:** last N turns (question, answer, sources), active topic, referenced entities, pending clarification.
- **Functions:** resolve pronouns and follow-ups ("What about its resolution?"), supply context to other agents, detect topic switches and reset the active topic so unrelated threads do not mix.
- **Boundary:** memory lives in the session store and is never indexed as document knowledge.

3.6 Agent interaction summary

Step	Agent	Reads	Produces	Next
1	Conversation Memory	session history	context window	Query Understanding
2	Query Understanding	query + context	classification, rewritten query	Clarification or Retrieval
3a	Clarification	classification	clarifying question	wait for user, then step 2
3b	Retrieval	rewritten query	chunks + scores	Response Generation
4	Response Generation	query + chunks	answer, sources, confidence	Orchestrator
5	Conversation Memory	final turn	updated session state	none

4. Knowledge Ingestion Pipeline

```
Upload > Validate > Extract > Clean > Chunk > Embed > Index > Update KB
```

Stage	What happens	Notes
1. Upload	User selects a file in the Knowledge Base page; frontend posts it to `POST /api/upload` as multipart form data	Show progress and errors
2. Validation	Check extension/MIME type (PDF, DOCX, TXT, CSV), size limit, non-empty file, duplicate check (hash)	Reject with a clear message
3. Parsing	PDF: page-wise text extraction; DOCX: paragraphs and tables; TXT: decode with encoding detection; CSV: row-wise conversion to readable text with header context	Libraries to be confirmed **[Verify]** (for example pypdf/PyMuPDF, python-docx, pandas)
4. Cleaning	Normalize whitespace and line breaks, remove repeated headers/footers where detectable, fix hyphenation, drop empty pages	Scanned PDFs without a text layer need OCR (limitation)
5. Chunking	Split into overlapping chunks, preferring paragraph/sentence boundaries	Configurable size and overlap (Section 9)
6. Embedding	Convert each chunk to a vector with the configured embedding model	Batch requests to reduce latency **[Verify model]**
7. Indexing	Store vector + metadata in the vector store; store document and chunk records in the metadata DB	Metadata: document ID/name, chunk ID, page/section, chunk index, character offsets, upload time
8. KB update	Mark the document `ready`; it appears in `GET /api/documents` and is searchable	On failure mark `failed` with error text and roll back partial chunks

Metadata handling. Every chunk carries enough metadata to render a citation (document name, page or section, chunk ID). The same IDs are used by the transparency panel so that displayed sources match retrieved chunks exactly.

Deletion. Deleting a document must remove its file, chunk records and vectors together to avoid orphaned retrieval results.

5. Query Resolution Pipeline

```
User input (text or voice)
  > POST /api/query {query, session_id}
  > Memory: load context
  > Query Understanding: classify, rewrite
  > [ambiguous?] > Clarification > (user reply) > re-classify
  > Retrieval: embed, Top-K, threshold filter
  > [no evidence?] > "not found" response + log as unanswered
  > Response Generation: grounded answer
  > Memory: store turn
  > Analytics: log query record
  > Response: answer + sources + confidence + timing
```

Response payload (recommended contract) [Verify]:

```
{
  "session_id": "s_123",
  "answer": "...",
  "query_type": "factual",
  "status": "answered | clarification_needed | no_evidence | error",
  "clarification_question": null,
  "sources": [
    {"document_name": "...", "chunk_id": "...", "page": 3, "score": 0.74,
"snippet": "..."}
  ],
  "confidence": {"level": "high | medium | low", "max_score": 0.74},
  "timing_ms": {"classify": 0, "retrieve": 0, "generate": 0, "total": 0}
}
```

Final presentation. The UI renders the answer, a collapsible Sources panel (document, page, snippet, score), a confidence badge, and optional text-to-speech playback. Clarification responses render as a question with optional quick-reply options.

6. Data Models and Storage Structures

The exact schema must be aligned with the codebase **[Verify]**. The following is the logical model.

6.1 Entity overview

Entity	Key fields
Document	`document_id` (PK), `filename`, `file_type`, `file_size`, `content_hash`, `status` (processing/ready/failed), `page_count`, `chunk_count`, `domain_tag`, `uploaded_at`, `error_message`
Chunk	`chunk_id` (PK), `document_id` (FK), `chunk_index`, `text`, `page` / `section`, `char_start`, `char_end`, `token_count`, `embedding` (in vector store)
Session	`session_id` (PK), `created_at`, `last_active_at`, `active_topic`
Message	`message_id` (PK), `session_id` (FK), `role` (user/assistant), `content`, `created_at`
QueryLog	`query_id` (PK), `session_id`, `query_text`, `rewritten_query`, `query_type`, `status`, `clarification_triggered`, `retrieved_chunk_ids`, `max_score`, `confidence_level`, `answered`, `input_mode` (text/voice), `latency_ms`, `created_at`
Setting	`key`, `value` (Top-K, threshold, chunk size, model name, TTS on/off)

6.2 Relationships

- One Document has many Chunks.
- One Session has many Messages and many QueryLogs.
- A QueryLog references the chunk IDs retrieved for that query.

6.3 Vector store record

```
{ id: chunk_id, vector: [float x D], metadata: {document_id, document_name, page, chunk_index} }
```

The vector store handles similarity search. The metadata database handles records, queries and analytics. Their concrete technologies (for example Chroma, FAISS, SQLite) must be stated from the actual dependencies **[Verify]**.

6.4 Analytics derivations

Analytics are computed from `QueryLog`:

- Total queries = count of rows.
- Category distribution = group by `query_type`.
- Unanswered = rows with `status = no_evidence`.
- Low-confidence = rows with `confidence_level = low` or `max_score` below the low-confidence threshold.
- Clarification rate = clarification-triggered rows divided by total.
- Top documents = frequency of document IDs in retrieved chunks.
- Knowledge gaps = clusters of repeated unanswered or low-confidence queries (Section 7.4).

7. Supporting Modules

7.1 Orchestration layer

The orchestrator is the single entry point behind `POST /api/query`. It loads memory, calls the agents in sequence, applies branching rules (clarify, no evidence, answer), measures per-stage latency, handles timeouts and retries with the language-model API, writes the QueryLog, and returns a uniform response. The exact order must match the implemented code **[Verify]**.

7.2 Voice modules

- **Speech-to-text:** browser `SpeechRecognition` (Web Speech API). Flow: microphone button, permission prompt, live transcript, user can edit, submit through the normal query path. Mark `input_mode = voice` in the log.
- **Text-to-speech:** `speechSynthesis` reads the answer when enabled, with play, pause and stop controls.
- **Handling:** permission denied, no speech detected, network error and unsupported browser each show a clear message. Support is browser-dependent (Chromium-based browsers are most reliable), so test in the target browser.

7.3 Response transparency

The Sources panel shows document name, page/section, chunk ID, snippet, relevance score and confidence badge. Displayed data is taken directly from the retrieval result of that query. A score is a similarity measure, not proof of correctness.

7.4 Analytics and knowledge-gap detection

- **Capture:** every query is logged with status, scores and latency.
- **Low-confidence rule:** `max_score` below a configurable threshold (higher than the retrieval cut-off) marks the answer low-confidence.
- **Unanswered rule:** no chunk above the retrieval threshold marks the query unanswered.
- **Gap detection:** group unanswered and low-confidence queries by topic (embedding similarity or keyword grouping **[Verify]**); topics that recur are listed as potential knowledge gaps with counts and example questions, with a suggestion to upload material on that topic.
- **Dashboard:** total queries, category chart, response-time trend, top documents, clarification rate, low-confidence list, unanswered list, knowledge-gap list.

8. API Endpoints

Base URL (local): `http://127.0.0.1:8000`. Frontend: `http://localhost:5173`.

Method	Endpoint	Purpose	Request	Response
GET	`/api/health`	Backend health	none	`{status: "ok"}` plus optional dependency checks
GET	`/api/documents`	List KB documents	none	Array of Document records with status and chunk counts
POST	`/api/upload`	Upload and process a file	multipart `file`	Document record, or error with reason
POST	`/api/query`	Ask a question	`{query, session_id?}`	Response payload (Section 5)
GET	`/api/analytics`	Aggregated metrics	optional date range	Metrics, unanswered list, gap list
GET	`/api/settings`	Current configuration	none	Settings object

Status codes: 200 success, 400 invalid input, 413 file too large, 415 unsupported type, 422 validation error, 500 server error, 503 model service unavailable. Confirm the behaviour of each endpoint by actual calls before describing it as operational **[Verify]**.

9. Configuration Parameters

Values below are recommended starting points to tune, not confirmed settings **[Verify]**.

Parameter	Purpose	Suggested start
`GEMINI_API_KEY`	Model access (environment variable, never committed)	none
`LLM_MODEL`	Generation model name	per current Gemini availability
`EMBEDDING_MODEL`	Embedding model	to be confirmed
`CHUNK_SIZE`	Characters or tokens per chunk	about 800 to 1,000 characters
`CHUNK_OVERLAP`	Overlap between chunks	about 10 to 20% of chunk size
`TOP_K`	Chunks retrieved	4 to 6
`RELEVANCE_THRESHOLD`	Minimum score to keep a chunk	tune on test queries
`LOW_CONFIDENCE_THRESHOLD`	Score below which an answer is flagged	tune on test queries
`MEMORY_TURNS`	Turns kept in session context	4 to 8
`MAX_UPLOAD_MB`	Upload size limit	for example 20
`CORS_ORIGINS`	Allowed frontend origins	`http://localhost:5173`
`VITE_API_BASE_URL`	Frontend API base URL	`http://127.0.0.1:8000`

10. Environment Requirements

- **OS:** Windows, macOS or Linux.
- **Python:** 3.10+ with a virtual environment; dependencies from `requirements.txt` (FastAPI, Uvicorn, document parsers, vector-store and database libraries, Gemini SDK **[Verify list]**).
- **Node.js:** current LTS with npm, for React/Vite.
- **Browser:** Chromium-based browser recommended for the Web Speech API.
- **Network:** internet access to the Gemini API.
- **Secrets:** API key in a `.env` file excluded from Git via `.gitignore`.

11. Deployment and Execution

Backend

```
cd backend
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install -r requirements.txt
# create .env with GEMINI_API_KEY and other settings
uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev          # serves http://localhost:5173
```

Verify startup: open `http://127.0.0.1:8000/api/health`, then the FastAPI docs page at `/docs`, then the frontend. File and module names above are placeholders to be matched to the repository **[Verify]**.

Production notes (optional): build the frontend with `npm run build`, serve behind a reverse proxy, run Uvicorn/Gunicorn workers, restrict CORS, use persistent volumes for the vector store and database, and add request timeouts, rate limiting and logging.

12. Suggested Project Structure

```
lumina-ai/
├── backend/
|   ├── main.py                 # FastAPI app and routes
|   ├── orchestrator.py         # pipeline controller
|   ├── agents/                 # query_understanding, retrieval, response,
clarification, memory
|   ├── ingestion/              # parsers, cleaner, chunker, embedder,
indexer
|   ├── storage/                # vector store and database access
|   ├── analytics/              # logging, aggregation, gap detection
|   └── config.py
├── frontend/
|   └── src/ (pages: Assistant, KnowledgeBase, Analytics, Settings;
components; voice hooks)
├── tests/
├── docs/
└── README.md
```